Ain Shams University

Faculty of Engineering

CSE331 – Data Structures and Algorithms

Colored Water Sort Solver Project

Student: Mahmoud Abdelaziz Mohammed

ID: 24p0485

Project Report

Recorded presentation is not included because it is optional according to the instructor.

1. Project Overview

The project is a console-based C++ solver for the Colored Water Sort puzzle. The program receives the initial tube arrangement and searches for a valid solution with the minimum number of moves. The project specification requires C++, supports a team size of one or two students, and states that a console interface is sufficient.

2. Team Responsibilities

Student	Responsibilities
Mahmoud Abdelaziz Mohammed	Implementation of the solver, data-structure design, testing, debugging, documentation, and

3. Data Structures and Why They Were Selected

vector> — Represents the complete puzzle. Each inner vector represents one tube and stores its layers from bottom to top.

struct State — Stores a puzzle state together with its parent state index and the source/destination tubes used to create it.

queue — Stores state indices for BFS. Because every pour is one move, BFS explores states in increasing move count.

vector of visited states — Keeps previously explored arrangements so the same state is not processed repeatedly.

4. Code Structure

Function / Component	Purpose
getTopColor()	Finds the top non-empty color of a tube.
getTopCount()	Counts the connected layers of the top color.
getEmptyPosition()	Finds the first empty position in a tube.
isSolved()	Checks whether every non-empty tube is full and contains one color.
canPour()	Checks whether a source-to-destination pour is valid.
pour()	Applies a valid pour to a copied state.
sameState()	Compares two puzzle arrangements.
isVisited()	Checks whether an arrangement was already explored.
solve()	Runs BFS and reconstructs the minimum-move solution.
printState()	Prints the final tube arrangement.

5. Algorithm

The solver uses Breadth-First Search. The initial arrangement is inserted into the queue. For each state, the program checks every source/destination pair, generates valid next states, ignores already visited states, and adds new states to the queue. Since each pour has a cost of one move, BFS explores states by increasing number of moves. Therefore, the first solved state found represents a minimum-move solution.

Each generated state stores the index of its parent state and the move that produced it. When a solved state is found, the parent links are followed backward and the moves are reversed to obtain the correct order.

6. Complexity Analysis

Let S be the number of reachable states, N the number of tubes, and C the tube capacity. Each state considers $O(N^2)$ possible source/destination pairs. State comparisons and the simple visited-state search can require $O(N \cdot C)$ work per stored state. With the vector-based visited structure, the worst-case visited checking can reach $O(S \cdot N \cdot C)$ for one generated state. Overall, the practical running time depends on the number of reachable states and can grow quickly for larger puzzles.

Memory usage is $O(S \cdot N \cdot C)$ for the stored states and visited arrangements, plus the BFS queue. The parent information is stored with each state so the solution path can be reconstructed without storing the full move sequence in every state.

7. Test Cases and Results

Test Case 1 — Official Example A

Input

```
Input: 4 / 2 / 1 2 / 2 1 / 0 0 / 0 0
```

Output / Result

```
Solution found. Minimum moves: 3. Moves: Tube 1 -> Tube 3; Tube 2 -> Tube 1; Tube 2 -> Tube 3. Final state: [1,1], [0,0]
```

Test Case 2 — Official Example B

Input

```
Input: 2 / 2 / 1 2 / 2 1
```

Output / Result

```
No solution exists.
```

Test Case 3 — Additional Test

Input

```
Input: 4 / 2 / 1 0 / 1 0 / 2 2 / 0 0
```

Output / Result

```
Solution found. Minimum moves: 1. Move: Tube 1 -> Tube 2. Final state: [0,0], [1,1], [2,2], [0,0].
```

8. GitHub Repository

GitHub repository link: TO BE ADDED AFTER CREATING THE PUBLIC REPOSITORY.

The repository should contain the C++ source file, sample inputs, README, and project documentation. The project specification requires the repository to be public and asks for meaningful development commits from both students when working as a pair.

9. Recorded Presentation

Not included. The instructor confirmed that the recorded presentation is optional. A live/demo-only approach can be used if required separately.

10. Conclusion

The final implementation provides a console-based Colored Water Sort solver using vector-based state representation, BFS, visited-state checking, and parent tracking. It solves the official solvable example in three moves, reports no solution for the official unsolvable example, and passes an additional test case.